TIKO — Technical Specification

Oct 10, 2026 · @ki

1. Overview and inputs

TIKO ships as one TypeScript codebase: a Next.js PWA that writes to the device first, syncs in the background, and runs one shared goal engine on both the client and the server. This spec turns the five source documents into build decisions for the MVP (weeks 1 to 8) and marks what V1 and V2 add.

Scale of the work. The Functional Requirements hold 93 requirements: 48 P0, 34 P1 and 11 P2, across 10 areas. The PRD adds 17 measurable non-functional targets (NFR-01 to NFR-17).

Design principles that drive every choice below

- Local first: every log is saved on the device before any network call, so speed and offline come free.
- One engine: goals, streaks, budget and XP are pure functions shared by client and server, so they cannot drift.
- Derive, don't increment: progress is recomputed from logs, so undo, edits and offline merges stay correct.
- Money is an integer in the smallest unit (paise); TIKO never moves or links to money.
- Sensitive data stays minimal: the journal is encrypted, analytics carry no text or amounts, and there are no ads.

Gaps in the inputs

- The Tech Stack document was not supplied. Stack choices here follow the dependencies named in PRD section 4 (Vercel, Neon or Supabase, Auth.js or Supabase Auth, Resend, Razorpay, Sentry) and are proposals to confirm. Where the Tech Stack document differs, it wins.
- The linked UI/UX design could not be opened, so colors, type and screen layouts are not restated here; see the Design Document.
- Several requirements conflict or are undefined; they are collected in section 10 as open questions, each with the default this spec assumes.

2. Architecture and tech stack

One Next.js app serves the PWA and its API from a single repository; the browser holds a full local copy of the user's data and treats the server as a sync target, not a dependency.

Layer	Proposed choice	Why
App framework	Next.js (App Router), React, TypeScript	One codebase for UI and API; matches the PRD's single-codebase aim
Styling	Tailwind CSS driven by design tokens from the Design Document	Tokens map to light and dark themes (FR-SET-01)
PWA	Service worker (Serwist) with web manifest	Install and open offline after first visit (FR-SYN-01, NFR-11)
Local store	IndexedDB through Dexie	Source of truth on the device; live queries drive the UI
Shared logic	`packages/engine`, pure TypeScript, no I/O	Same goal, streak, budget and XP code on client and server
API	Next.js route handlers on Vercel, Zod validation	Serverless, free tier to start
Database	Postgres on Neon or Supabase, Drizzle ORM	Relational fits goals, logs and budgets; export is straightforward
Auth	Auth.js (magic link plus Google), or Supabase Auth if Supabase is chosen	No passwords (A3); pick one provider, not both
Email	Resend	Magic links and reminder fallback
Push	Web Push (VAPID)	Reminders where permitted (FR-NOT-03)
Scheduler	External cron such as Upstash QStash or Supabase pg_cron	Reminders need roughly 5-minute precision; check current Vercel cron limits for your plan
Object storage	Vercel Blob or S3-compatible bucket	Treat photos

Layer	Proposed choice	Why
Payments (V1)	Razorpay	India-first billing
Monitoring	Sentry, plus an uptime monitor	NFR-14 and NFR-03
Testing	Vitest, fast-check, Playwright, Lighthouse CI, axe	See section 9

Request flow for one log

1. The user taps a habit card; the UI calls the engine and writes the log plus recomputed goal progress to IndexedDB in one transaction (target: card updates in under 100 ms, saved on device in under 300 ms, NFR-02).
2. The same transaction adds a mutation to the outbox; the UI shows "Saved on this device" until the outbox is empty.
3. When online, the sync client sends the outbox to the server in a batch; the server applies each mutation once, reruns the engine and returns the new state and a cursor.
4. Other devices pull changes since their last cursor.

Capacity note (approximate). At 50,000 users (NFR-12), assuming 30% daily active and about 12 rows written per active user per day, the database grows by roughly 180,000 rows a day, about 66 million a year. Postgres handles this with an index on (user_id, local_date), but free-tier storage will run out long before that; plan a paid database before public launch.

3. Data model

Every user-owned row carries a client-generated UUID, `user_id`, `updated_at`, `deleted_at` (a tombstone, so deletes sync) and a server `seq` for the change feed; the same shapes live in IndexedDB and Postgres.

Conventions

- Money: integer paise (smallest unit) plus a `currency` code on the user, default INR (FR-MON-03).
- Dates: each log stores `local_date` (YYYY-MM-DD in the user's time zone at log time) beside a UTC timestamp, so a later time-zone change never rewrites history. A day runs midnight to midnight local.

- Plan limits: free-plan caps live in an entitlements config, not in code paths, so V1 billing only changes config.
- Journal body: encrypted before it is stored (see section 7); title-less, one body field.

Entity	Key fields	Notes
user	id, email, name, time_zone, currency, plan, softened_until, created_at	One row per account; guests have no server row
habit	id, core_key (nullable), name, icon, mode (tap, check, timer, number), unit, target, kind (build, limit), cadence (daily, weekly_total), paused_until, archived_at, sort_order	Unique (user_id, core_key) so the 7 core habits never duplicate on merge
habit_log	id, habit_id, value, local_date, logged_at, source	Append-mostly; edits allowed for 7 days (FR-HAB-10)
goal	id, type (daily_target, streak, weekly_total, count, limit, savings), habit_id or category_id, target, start_date, end_date, status, progress, nudged_50, nudged_90, treat_id, softened_factor, claimed_at	Status: active, due, completed, claimed, archived. There is no failed state
treat	id, name, photo_ref, cost, kind (free, paid), is_starter	30 starter treats ship as static data
expense	id, amount, category_id, local_date, note, recurring_id, treat_id (nullable)	Paid treats are flagged so they stay out of ordinary spending (FR-TRT-06)
category	id, name, is_custom, monthly_limit	Eight defaults (FR-MON-02)
budget	id, month, amount, treat_budget	One monthly budget (FR-MON-05)
recurring_expense	id, amount, category_id, day_of_month, active	Posts via scheduler (FR-MON-09, P1)

Entity	Key fields	Notes
mood_entry	id, score (1 to 5), energy (low, medium, high), tags (max 3), note (one line), local_date, slot	One per check-in time (FR-MND-03)
journal_entry	id, body_ciphertext, nonce, key_version, local_date, kind (free, gratitude)	Plaintext never reaches logs or analytics
xp_event	id, source_type, source_id, points	Append-only ledger; XP is never removed (FR-ENG-04)
notification_pref	reminder_time, tone, quiet_start, quiet_end, push_on, email_on	Defaults 22:00 to 08:00
push_subscription	id, endpoint, keys, last_ok_at	Deleted on a 410 response
notification_log	id, kind, sent_at, channel	Enforces 2 a day and the 7-day re-engagement gap
device	id, user_id, last_seq	Supports pull cursors

Indexes. (user_id, local_date) on habit_log, expense and mood_entry; (user_id, seq) on every synced table; (user_id, status) on goal.

4. API design

The API is small because the client owns reads: two sync routes carry almost all traffic, and the rest handle identity, notifications, export and deletion.

Route	Method	Purpose	Requirements
/api/sync/push	POST	Apply a batch of mutations; idempotent by `op_id`; returns authoritative rows and a cursor	FR-SYN-02, 03, 04, 06
/api/sync/pull	GET	Rows changed since `cursor` (by `seq`), paged	FR-SYN-03
/api/sync/merge-guest	POST	Upload a guest's device data on first sign-in; same rules as push	FR-ACC-03

Route	Method	Purpose	Requirements
/api/auth/*	various	Magic link, Google OAuth, session, sign-out	FR-ACC-02, 04
/api/me	GET, PATCH	Name, time zone, currency, entitlements	FR-ACC-08
/api/push/subscribe	POST, DELETE	Register or remove a push subscription	FR-NOT-03
/api/export	GET	CSV and JSON of all data, including mind data	FR-SET-02
/api/account	DELETE	Confirm and start deletion	FR-SET-03
/api/cron/*	POST	Reminders, recurring expenses, deletion purge, recap; secret-protected	FR-NOT-*, FR-MON-09
/api/billing/*	POST	Razorpay order and webhook (V1)	FR-SET-06

Rules for every route

- Validate input with Zod; reject unknown fields. Scope every query by the session's `user_id`.
- Idempotency: each mutation carries a client `op_id`; the server stores applied ids for 30 days and ignores repeats, so a retried batch never duplicates a log.
- Conflicts: row-level last write wins on `updated_at`; the server rejects client timestamps more than 5 minutes in the future and clamps them. A delete is a tombstone and obeys the same rule.
- Rate limits (NFR-05): sign-in 5 requests per 10 minutes per email and 20 per hour per IP; write routes 120 per minute per user. Use a small Redis-style store, not the main database.
- Errors: JSON `{code, message}` with standard status codes; sync errors are retryable and never cause the client to discard local data.
- Magic links: a random 32-byte token, stored hashed, valid 15 minutes, marked used on first click (FR-ACC-04).

5. Core engines

All rules live in `packages/engine` as pure functions that take logs, goals and settings and return new state plus events (nudge, completion, level-up). The client runs them on every log; the server reruns them when it applies a push, and the server result wins.

Goal engine (FR-GOL-04 to 08, UC5)

- Trigger: after every log, edit, undo or delete, recompute all active goals that use that habit or expense category. Goals are capped at 5, so this is a handful of small queries over a short window and fits the 100 ms budget.
- Progress is derived from logs, never incremented, so undo (10 seconds, FR-HAB-03) and edits (7 days, FR-HAB-10) stay correct.
- Per type: daily_target counts days whose total meets the target; streak counts days under the streak rule below; weekly_total sums the week; count counts qualifying logs; limit counts closed days with spend at or under the cap; savings sums contributions.
- A limit-goal day only counts once it has ended, because spend can still rise; the engine evaluates open days on the next app open and from the scheduler.
- Nudges fire once at 50% and once at 90%, tracked by `nudged_50` and `nudged_90` (FR-GOL-06).
- When progress reaches target the goal becomes completed in the same transaction and the celebration shows at once. If the completing log is undone within 10 seconds and the treat is unclaimed, the goal returns to active.
- A deadline passing sets status to due and offers extend, shrink or restart; no code path writes a failed state (FR-GOL-08).

Streak rule (FR-ENG-01, F1)

A day is met, missed or paused (paused days are skipped). Today is never a miss until it ends.

```
for each day in order:
  if met:      streak += 1
  if paused:   continue
  if missed:
    if previous day was missed:        streak = 0       # two in a row
    elif forgiven_within_last_6_days:  streak = 0       # second miss in
window
    else:                              mark forgiven    # half-filled dot
```

The forgiven-within-last-6-days branch is an interpretation: the documents say one forgiven day "in any 7 days", which this spec reads as a second, non-consecutive miss inside the window also breaking the streak. See open question 3.

Budget (FR-MON-05, 06, 07)

```
\text{safe to spend} = \max\left(0,\ \left\lfloor \frac{\text{monthly budget}
- \text{spent this month}}{\text{days left, including today}}
\right\rfloor\right)
```

Spent excludes paid treats and income. A category warning shows once per category per month when spend first reaches 80% of its limit; entries are never blocked.

XP and levels (FR-ENG-04, 05)

XP is an append-only ledger. Only four values are defined (water glass 1, workout 20, journal 15, mood check-in 5); the rest, and level names and thresholds, are config and need product input.

Low-mood softening (FR-MND-08 to 10)

- Trigger: three consecutive days with a mood score of 1 or 2 (assumed meaning of "low").
- Effect: set `softened_until`, scale goal targets by `softened_factor` (assumed 25% lower, configurable), hide streak counters, show the support card once.
- Dismissal hides the card for 7 days unless another low mood is logged. The user can turn softening off in Me.
- Self-harm wording: detected on the device while the user types, against a maintained word list, so no plaintext is sent for this purpose. It shows the same calm card and never locks the user out. Copy never diagnoses.

Engine tests. Property tests (fast-check) for streak and goal recompute: result is identical after shuffled log order, after undo then redo, and after a day-boundary change in time zone.

6. Offline storage and sync

IndexedDB is the source of truth on every device; the server is a copy that lets one account span devices. A log is safe the moment the local transaction commits (NFR-04).

- **Write path:** UI, engine, then one Dexie transaction that stores the log, the recomputed goal rows and an outbox entry `{op_id, entity, id, patch, updated_at, device_id}`. The badge "Saved on this device" shows while the outbox has entries (FR-SYN-03).
- **When sync runs:** on the browser online event, on app open and tab focus, every 30 seconds while open and online, and through Background Sync where the browser supports it. Failed pushes retry with backoff capped at 30 seconds, which keeps the "within a minute of reconnecting" promise (F5).
- **Never lose data:** local rows are deleted only by a user action or after the server confirms a tombstone. A failed or rejected sync keeps the outbox entry and the local row (FR-SYN-06).
- **Conflicts:** the latest `updated_at` wins per row (FR-SYN-04). Habit logs are append-mostly, so real conflicts are rare.
- **Completed offline:** the device engine shows the celebration at once (FR-SYN-05). When the server reruns the engine and disagrees, the client adopts the server state quietly; a claimed treat is never taken back and no failure wording is shown.
- **Timers:** store `started_at` and compute elapsed time from it, so closing the tab loses nothing (FR-HAB-08).
- **Service worker:** precache the app shell and fonts; the first visit must finish loading before offline works (NFR-11).

Guest mode and merge (FR-ACC-01, 03)

Guests store everything locally with no server row. On first sign-in the device uploads its data through merge-guest. Because ids are client-generated UUIDs the upload is idempotent, and core habits match on `core_key`, so nothing duplicates. If the account already has data, the two sets are unioned and conflicting rows resolve by latest `updated_at`.

Browser storage risk. Browsers may evict site data for guests who never install the app or return; Safari is the strictest. Mitigations: call `navigator.storage.persist()`, prompt installation, show the sign-in prompt after the first completed goal (FR-ACC-07), and keep export available. Guest data loss is the one place NFR-04 cannot be fully guaranteed by the app alone, so the onboarding copy should say guest data lives on this device.

7. Auth, notifications, security and privacy

Sessions. HttpOnly, Secure, SameSite=Lax cookies with a 30-day sliding expiry and a 90-day absolute limit (proposed values). Sign-out revokes the session server-side.

Notifications (FR-NOT-01 to 06)

- Permission is requested only after the user sets a reminder time, never on first load (FR-ACC-06).
- A scheduler job runs about every 5 minutes, finds users whose reminder time falls in the window in their time zone, and sends by web push; if there is no subscription or push fails, it sends email. A 410 response removes the subscription and switches that user to email.
- Limits: at most 2 notifications a day and none inside quiet hours (default 10 pm to 8 am), enforced from `notification_log`. After 3 days without a log, one re-engagement message, then none for 7 days.
- Copy comes from a tone library (gentle, playful, minimal). Payloads never contain journal text or exact amounts. A unit test scans the library for banned words (failed, lazy, behind).
- Proposed, not in the documents: skip a reminder when that day's ring is already closed.

Security (NFR-05)

- HTTPS only with HSTS; a strict Content Security Policy; CSRF protection through SameSite cookies plus an origin check on writes.
- Authorization by `user_id` on every query; use Postgres row-level security if Supabase is chosen.
- Secrets only in environment variables; dependency and secret scanning in CI; an OWASP ZAP baseline scan before beta.
- A written security checklist and review is a release gate for the public MVP.

Journal encryption (FR-MND-04, 05; NFR-06)

Each user has a data key, wrapped by a master key held in the hosting provider's secret store or a KMS. Bodies are encrypted with AES-256-GCM before they are written to the database, and journal search runs on the device over the local copy, so the server never needs plaintext to search. This is encryption at rest, not end-to-end: the operator could decrypt. True end-to-end encryption would break magic-link account recovery and is not planned (open question 11). The local IndexedDB copy is not encrypted by the app, so the help page should say so.

Privacy

- Analytics use an allowlist of event names and enum properties; no journal text, notes or exact amounts (NFR-06). Mind data is never used for ads and never shared.
- Export covers every table including mind data as CSV and JSON (FR-SET-02).
- Deletion removes access immediately and purges all rows within 30 days (NFR-07). Database backups must also age out within that window, so set backup retention to 30 days or less.
- DPDP Act 2023 and GDPR review, with legal advice, before public launch (NFR-16): consent text, data location, a grievance contact, and how mood data is classed.
- Support card: helpline lists are per-country config; the countries are an open decision.

8. Frontend

The Today screen must be interactive in under 2 seconds on a mid-range phone over 4G (NFR-01), so it is rendered from the cached shell and IndexedDB, not from a server round trip.

- **Routes:** onboarding, Today, Goals (with a goal page), Money, Mind and Me, plus a full-screen breathing view. Exact navigation and layouts follow the Design Document.
- **State:** Dexie live queries are the data layer; a small store (Zustand) holds only UI state such as open sheets and undo timers. No server-state cache library is needed for core screens.
- **Tokens and themes:** colors, type, spacing and module colors come from the Design Document as CSS variables; light, dark and device theme switch by variable set (FR-SET-01).
- **Performance budget (proposed):** initial JavaScript for Today at or under 170 KB gzipped; Money, Mind and Goals code-split; confetti loaded only when reduced motion is off; fonts subset with a system fallback; Lighthouse CI fails the build when Today's interactive time exceeds target on a throttled mid-range profile.
- **Treat photos:** resize on the device to at most 1280 px WebP, store locally, upload when online. Starter treats ship with bundled images.
- **Accessibility (NFR-08, 09):** 4.5:1 text contrast, 48 px tap targets, visible focus, labels on icon-only buttons, no meaning by color alone, reduced-motion fallbacks (fade instead of confetti, static breathing guide). Axe runs in CI and a screen-reader pass is a beta gate.

- **Undo:** a single shared 10-second undo mechanism serves habit logs and expenses (FR-HAB-03, FR-MON-04).
- **Browser support:** latest two versions of Chrome, Safari, Edge and Firefox, from 360 px width (NFR-10). Web push on iOS Safari works only for installed PWAs, so email must carry reminders for other iOS users.
- **Localisation (NFR-15):** all strings in message files and all money and dates through Intl, so a second language is a content task, not a refactor.

9. NFR mapping, testing and observability

Each PRD target has a technical approach and a named check. Rows marked beta are required for the MVP beta release (NFR-01, 02, 04, 05, 06, 08, 11).

NFR	Approach	Verified by	Beta gate
01 Today under 2 s	Cached shell, IndexedDB render, code splitting	Lighthouse CI plus real-device test	Yes
02 Tap 100 ms, save 300 ms	Optimistic local write; "saved" means committed on the device	Performance trace	Yes
03 99.5% availability	Serverless hosting; the app works offline when the server is down	Uptime monitor	No
04 No log lost	Outbox, tombstones, no destructive sync	Offline and failure tests	Yes
05 Security	HTTPS, sessions, rate limits (section 7)	Checklist and review	Yes
06 Privacy	Encrypted journal, analytics allowlist	Code review and data audit	Yes
07 Export and deletion	Export route, deletion purge, backup retention	End-to-end test	V1
08 WCAG 2.2 AA	Tokens meet contrast; focus and labels in components	Axe plus screen-reader pass	Yes
09 Motion and color	Reduced-motion branch; icons plus color	Manual check	No

NFR	Approach	Verified by	Beta gate
10 Browsers, 360 px	Responsive layout; no unsupported APIs without fallback	Cross-browser test	No
11 Offline	Service worker, local-first writes	Airplane-mode test	Yes
12 50,000 users	Indexes, paged sync, capacity plan (section 2)	Load test	No
13 Test coverage	Engine in a pure package	CI report	No
14 Observability	Sentry with scrubbing; allowlisted events	Dashboard review	No
15 Localisation	Message files and Intl	Manual check	No
16 Compliance	DPDP and GDPR review	Legal sign-off	Public MVP
17 Content	Banned-word test; disclaimers in onboarding and Settings	Content review	Yes (disclaimer text)

Test strategy

1. Unit and property tests for the engine (streaks, goals, budget, XP), the layer NFR-13 names.
2. Integration tests for sync routes against a real Postgres: duplicates, retries, conflicts, guest merge, deletion.
3. End-to-end tests in Playwright for the critical flows: first goal and treat (UC1), expense and safe-to-spend (UC2), offline then sync (UC3, with offline mode switched on in the browser), low-mood card (UC4), claim (UC5).
4. Cross-browser runs on Chromium, WebKit and Firefox; Edge is covered by Chromium.
5. Accessibility with axe in CI; the manual screen-reader pass before beta.
6. A k6 load test of the sync routes for NFR-12, with the target marked to validate.
7. CI on every pull request (typecheck, lint, tests, Lighthouse budget); main deploys to a preview, then production.

Observability. Sentry captures errors with request bodies and journal fields scrubbed. Product analytics follow the allowlist in section 7. Server logs hold ids and status codes, never content. Dashboards track the PRD success measures: goal completed within 7 days (40%), day-7 retention (35%), day-30 retention (20%), days logged per week (4 or more), and seconds to first log (under 10).

10. Build order, risks and open questions

The MVP is built in eight weekly steps, each ending in something testable; the PRD says durations stretch if one person builds part-time. V1 (weeks 9 to 16) adds savings goals, the weekly recap, health-app step import and the Plus plan; V2 (weeks 17 to 26) adds a native wrapper, more languages and coach-led challenges.

Week	Build	Requirements covered
1	Repo, CI, schema, auth (magic link, Google), guest mode, PWA shell, design tokens	FR-ACC-01, 02, 04
2	Dexie store, engine package, habits and Today screen with four tracking modes	FR-HAB-01 to 05
3	Outbox, sync routes, guest merge, offline behavior	FR-SYN-01 to 04, 06; FR-ACC-03
4	Goal engine, templates, treats, celebration, claim, Trophy shelf	FR-GOL-01 to 08; FR-TRT-01 to 05
5	Streaks, daily ring, forgiving rule; expense logging, budget, safe-to-spend	FR-ENG-01, 08; FR-MON-01 to 06, 13
6	Mood check-in, encrypted journal, disclaimer; support card if time allows	FR-MND-01, 02, 04, 05, 13, 14
7	Reminders, onboarding, settings, export and deletion	FR-NOT-01 to 03, 06; FR-ACC-05, 06
8	Hardening: accessibility, performance, security review, beta gate checks	NFR-01, 02, 04, 05, 06, 08, 11

P1 requirements fill gaps within these weeks; recurring expenses, XP and levels, and the low-mood card (all P1) are the first to move to V1 if time runs short, because they add no step to the core log, goal, treat loop.

Top technical risks

Risk	Response
Browsers evict guest data	Persistent storage request, install and sign-in prompts, export
Reminder timing on free-tier cron	Use an external scheduler; verify limits before beta
Clock skew breaks last-write-wins	Clamp future timestamps; log-style data is mostly append-only
Scope grows across four modules	Hold the P0 list; engine stays one package
iOS web push limits	Email fallback; installed-PWA prompt

Open questions and assumed defaults. Each default is what this spec assumes until someone decides.

1. Database and auth: Neon with Auth.js, or Supabase for both? Assumed: Neon, Drizzle and Auth.js, because it is easiest to move away from.
2. Reminder scheduler: which service? Assumed: an external cron with 5-minute precision.
3. Streak rule: does a second, non-consecutive miss inside 7 days break the streak? Assumed: yes (strict reading).
4. FR-GOL-01 (P0) lists six goal types including savings, but savings goals are P2 and V1. Assumed: the type exists in the schema, the MVP ships five types.
5. FR-SET-06 says Plus unlocks unlimited goals, but FR-GOL-03 caps active goals at 5 on any plan. Assumed: 5 is a hard cap.
6. FR-HAB-04 provides 7 core habits while the free plan allows 5 habits. Assumed: the 7 are a catalog and the active count is capped by plan.
7. FR-TRT-01 (P0) allows a treat photo, while FR-SET-06 gates custom treat photos behind Plus. Assumed: photos are open in the MVP and gated when billing arrives.
8. "Low mood" and the softening size are undefined. Assumed: a score of 1 or 2, and targets 25% lower.
9. XP values exist for only four actions, and level names and thresholds are missing.
10. Where a user sets the treat budget used by paid treats (FR-TRT-06) is not specified.
11. Journal encryption: server-side at rest (assumed) or end-to-end?
12. Which analytics tool, and the list of countries and helplines for the support card.

13. The PRD shows all open decisions as ticked (free-plan limits, recurring expenses, theme, trademark, Plus price) without recording the outcomes, and the Functional Requirements still list several as open. Assumed: the Functional Requirements priorities stand, and the Plus price waits for beta data.

Write acceptance tests for every P0 requirement before development starts; the Functional Requirements document already asks for this.